Cross-Residual Graph Neural Networks for Biomolecular Graph Representation Learning

Your Name
Department/Institution
University/Organization
City, Country
email@example.com

April 5, 2026

Abstract

Reliable representation learning on biomolecular graphs remains difficult because repeated message passing can wash out local structural cues while simple identity skips do not explicitly model complementary pathways. This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a compact family of graph classification architectures that reuses intermediate states either within a branch, across parallel branches, or through graph-level summaries passed between sequential blocks. The goal is not to claim that every cross-residual design is universally superior, but to determine when cross-path information exchange is useful relative to both plain stacking and stronger residual reuse. We evaluate plain, residual, and cross-residual variants under a unified stratified 5-fold protocol with validation-based early stopping, learning-rate scheduling, gradient clipping, and TensorBoard-backed analysis logging. The final study covers seven datasets, with PROTEINS, DD, and ENZYMES forming the topic-facing biological core and MUTAG, AIDS, Mutagenicity, and MSRC_9 serving as broader robustness checks. The resulting empirical picture is selective rather than absolute: residual variants remain the strongest default family, winning four of the seven datasets, whereas cross-residual variants win three datasets and outperform the plain baseline on six of seven. These results position CR-GNN as a practical design space for biomolecular graph representation learning, with dataset-dependent cross-residual gains and a clear path toward future integration with richer omics-derived signals.

1 Introduction

Graph neural networks (GNNs) are now widely used for structure-aware learning on molecules, proteins, and other biological interaction systems. In biomolecular settings, graph representations offer a natural way to encode local neighborhood geometry together with larger structural context, which is essential for tasks such as protein family discrimination, enzyme-related classification, and molecular activity prediction. However, these same settings also expose a persistent weakness of deep message passing: repeated aggregation can blur discriminative local patterns, collapse intermediate diversity, and make optimization unstable when graph labels depend on coordinated substructures rather than on a single dominant motif.

This problem is especially relevant when the target application is biological rather than purely synthetic. Protein-related graphs often contain multiple structurally meaningful regions whose interaction matters for the graph label. A model that only stacks message-passing layers sequentially may underuse such complementary information, while a simple residual skip only re-injects the same branch history without testing whether a second pathway or a graph-level feedback signal can preserve more useful context. This motivates a more explicit study of cross-residual information flow.

Modern GNN research has proposed several mechanisms to improve deep training, including residual connections [1,2], normalization [3,4], edge dropout [5], and multi-scale readout designs such as Jumping Knowledge [6]. These methods improve optimization or preserve multi-depth features, but most still operate inside a single branch. In parallel, graph-level pooling and hierarchical modules improve global summarization, yet they do not directly test whether exchanging intermediate states across branches or reusing graph summaries across block boundaries yields a better trade-off for biomolecular graph inference.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a unified family of graph classification models that isolates three kinds of information reuse: intra-branch residual accumulation, cross-branch node-state exchange, and graph-level summary propagation across sequential blocks. Rather than presenting one oversized architecture, we formulate a controlled model family that includes plain, residual, and cross-residual variants under the same message-passing backbone. This makes it possible to ask a more useful question than whether one model wins everywhere: when does cross-residual interaction help beyond a simpler residual baseline on biomolecular graphs, and when does residual reuse remain the better default?

The target venue also shapes the framing of this study. Our current experiments do not constitute a full integrative omics pipeline, nor do they directly solve novel gene or motif discovery. The contribution is narrower and more defensible: we provide a graph representation architecture that is empirically evaluated with a protein-oriented main benchmark and a broader full-suite robustness study, and is designed to preserve multi-scale biological structure more effectively than plain stacking. This positions the method as a biomolecular graph learning backbone that can later be extended toward richer omics-derived inputs.

The contributions of this work are summarized as follows:

- We formalize CR-GNN as a compact architecture family that separates plain propagation, node-level residual reuse, node-level cross residual exchange, graph-level residual propagation, and graph-level cross residual interaction under a shared message-passing pipeline.
- We introduce a stabilized experimental protocol that combines a protein-oriented main benchmark with a full-suite robustness evaluation, using stratified 5-fold evaluation, inner validation splits, learning-rate scheduling, and gradient clipping to avoid the collapse behavior observed under weaker training settings.
- We show that cross-residual design is useful but dataset-dependent: it is not a universal replacement for stronger residual baselines, yet it remains competitive, consistently improves over plain stacking in most datasets, and can outperform residual reuse on selected

graph regimes.

- We provide practical analysis artifacts, including TensorBoard signals, embedding statistics, and exportable evaluation outputs, so that architectural claims can be inspected beyond a single accuracy table.

2 Related Work

We review related work from five perspectives: graph neural networks for graph classification, message passing architectures and operators, deep GNN training challenges, hybrid architectures, and cross-layer aggregation mechanisms. Our work builds upon these foundations by introducing a unified framework for comparing multiple residual information-flow patterns under a shared graph classification setting.

2.1 Graph Neural Networks for Graph Classification

Graph-level prediction requires aggregating node-level features into fixed-size graph representations. Early graph neural networks [7] laid the foundation for learning on graph-structured data through recursive message passing. The Message Passing Neural Networks (MPNN) framework [8] unified various GNN architectures under a common formalism, identifying message and vertex update functions as core components.

For graph classification, readout functions play a crucial role in converting node embeddings to graph-level representations. Simple approaches apply global pooling operations such as mean or sum pooling [9]. However, these methods may not adequately capture hierarchical structural information. To address this, differentiable pooling methods have been proposed: DiffPool [10] learns node cluster assignments to coarsen graphs hierarchically, SAGPool [11] employs self-attention to identify salient nodes, and LocalPool [12] enables sparse hierarchical classification. Recent surveys provide comprehensive overviews of graph pooling techniques [13, 14]. While these approaches achieve strong performance, they primarily focus on pooling design rather than on how residual information should be reused across node-level and graph-level computation stages.

2.2 Message Passing Architectures and Operators

Various message passing operators have been developed, each imposing different inductive biases on neighborhood aggregation. Graph Convolutional Networks (GCN) [15] approximate spectral convolutions through normalized adjacency matrices, performing low-pass filtering that smooths node features. GraphSAGE [16] samples and aggregates features from local neighborhoods using mean, LSTM, or pooling aggregators, enabling inductive learning on unseen graphs. Graph Isomorphism Networks (GIN) [17] use injective aggregation functions to achieve maximum discriminative power, matching the expressive power of the Weisfeiler-Lehman test.

Attention mechanisms have been widely incorporated to weight neighbor contributions adaptively. Graph Attention Networks (GAT) [18] compute attention coefficients for each neighbor, allowing nodes to focus on relevant information. Personalized Graph Neural Networks

(PGNN) [19] further extend attention mechanisms for personalized aggregation. Recent work generalizes Transformer architectures to graphs [20] and incorporates edge directionality [21,22] for improved modeling of directed and heterophilic graphs.

Other operator variants include ARMA filters [23] for flexible frequency response, MixHop [24] for mixing multi-hop neighborhoods, and anti-symmetric convolutions [25] for stable deep architectures. While these operators are typically studied and deployed individually within homogeneous architectures, a critical gap remains: the literature provides much less comparison of how different residual information-flow mechanisms behave across operator families under a unified training protocol.

2.3 Deep GNN Training and Representation Degradation

Training deep GNNs faces fundamental challenges due to representation degradation. The *over-smoothing* phenomenon causes node representations to converge toward indistinguishable values as depth increases [26–28]. Theoretical analysis [28] provides non-asymptotic bounds on over-smoothing, revealing fundamental limitations in expressive power for deep architectures. Additionally, *over-squashing* [29,30] occurs when long-range dependencies cannot be effectively propagated due to graph bottlenecks, further constraining model performance.

To enable deep training, various techniques have been proposed. Residual connections [1], originally developed for computer vision, have been adapted to GNNs through residual gated graph convolutions [2]. Normalization methods such as PairNorm [3] and GraphNorm [4] alleviate over-smoothing by normalizing node features, while learnable normalization [31] adapts to different graph distributions. Regularization techniques like DropEdge [5] randomly drop edges during training to prevent overfitting and improve generalization.

Advanced architectures attempt to train extremely deep networks: techniques for training 1000-layer GNNs [32] and simplified deep GCN approaches [33] demonstrate that depth can improve performance with proper design. However, these methods primarily focus on single-path deepening and do not explicitly compare node-level residual reuse with graph-level residual propagation as two distinct ways of preserving information at depth.

2.4 Multi-Operator and Hybrid Architectures

Combining multiple aggregation schemes or architectural components has shown promise in enhancing representational capacity. Multi-hop attention [34] enables information flow across longer distances, while ARMA filters [23] provide flexible frequency responses beyond polynomial filters. Hybrid architectures incorporate edge directionality [21, 22] to capture directed relationships, and anti-symmetric convolutions [25] improve stability in deep networks.

Surveys on attention-based GNNs [35] and graph embeddings [36] provide comprehensive taxonomies of architectural variants. DenseGNN [37] connects all layers densely, similar to DenseNet in computer vision, to maintain information flow. Hierarchical pooling methods [10, 38] create multi-scale representations through graph coarsening. However, most existing hybrid approaches emphasize component combination rather than a systematic comparison of where residual information should be injected and reused. Our work addresses this gap

by contrasting node-level residual reuse, graph-level residual propagation, and cross-branch information exchange within a unified architecture family.

2.5 Cross-Layer and Multi-Scale Aggregation

Cross-layer connections aim to capture multi-scale information by aggregating features from different depths. Jumping Knowledge (JK) networks [6] adaptively select which layer's representation to use for each node, enabling flexible neighborhood range selection. Graph U-Net [39] adopts encoder-decoder architecture with skip connections between corresponding layers.

Propagation-based methods improve information flow across multiple hops: APPNP [40] combines personalized PageRank with neural predictions, while PairNorm [3] and DropEdge [5] mitigate over-smoothing through normalization and regularization. Recent work on over-smoothing [27, 41] provides comprehensive analysis and solutions.

Despite these advances, most cross-layer methods operate within a single homogeneous branch using one type of operator. A fundamental limitation is that they rarely distinguish between residual reuse at the node-representation level and residual reuse at the graph-summary level. Our framework focuses on this distinction by introducing node-level and graph-level cross-residual mechanisms within a common graph classification design space. This perspective enables a more direct comparison of how different information-flow patterns affect robustness, depth tolerance, and graph-level discrimination.

3 Task Definition

We formally define the graph classification problem and establish notation for our cross-residual framework. Rather than binding the method to a single message-passing operator, we treat the operator choice as a modular backbone and study how different residual information-flow mechanisms affect graph classification behavior under a common formulation.

3.1 Graph Representation and Notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, where d_{in} is the input feature dimension. The node features for the entire graph are collected in a matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$, where the i-th row corresponds to $\mathbf{x}_i$.

The graph structure is represented using an adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ if there exists an edge $(v_i, v_j) \in \mathcal{E}$, and $A_{ij} = 0$ otherwise. For undirected graphs, the adjacency matrix is symmetric ($\mathbf{A} = \mathbf{A}^\top$). Optionally, edge features can be represented as $\mathbf{e}_{uv} \in \mathbb{R}^{d_e}$ for each edge $(u, v) \in \mathcal{E}$, or collected in a tensor $\mathbf{E} \in \mathbb{R}^{n \times n \times d_e}$.

3.2 Graph Classification Problem

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^{N}$ consisting of N graphs, where each graph $\mathcal{G}_i$ is associated with a label $y_i \in \mathcal{Y}$, our objective is to learn a mapping function $f_\theta : \mathcal{G} \to \hat{y}$ that predicts the label $\hat{y} \in \mathcal{Y}$ for an unseen graph. The label space $\mathcal{Y}$ depends on the task type:

- **Binary classification**: $\mathcal{Y} = \{0, 1\}$ (e.g., molecular toxicity prediction)
- **Multi-class classification**: $\mathcal{Y} = \{1, 2, \ldots, C\}$ (e.g., graph categorization into C classes)
- **Multi-label classification**: $\mathcal{Y} \subseteq \{1, 2, \ldots, C\}$ (e.g., a graph may belong to multiple categories simultaneously)

The function f_θ is parameterized by neural network weights θ and operates by first learning node-level representations through message passing, then aggregating these into a graph-level representation, and finally mapping this representation to the output label space.

3.3 Message Passing with Multiple Operators

Our framework supports multiple message passing operators, where each operator imposes a distinct inductive bias on neighborhood aggregation. Common operators include:

- **GCN** [15]: Normalized adjacency-based aggregation with low-pass filtering
- **GAT** [18]: Attention-weighted aggregation with learnable importance coefficients
- **GraphSAGE** [16]: Sampling-based aggregation with mean/LSTM/pooling operators
- **GIN** [17]: Injective aggregation for maximum discriminative power
- **Transformer** [20]: Self-attention over nodes for long-range dependencies

For a chosen operator Φ, node representations are updated over L layers. At layer ℓ, the representation $\mathbf{h}_i^{(\ell)}$ of node v_i is computed as:

$$\mathbf{h}_i^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_\Phi\left(\left\{\mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i)\right\}, \mathbf{h}_i^{(\ell)}\right) + \mathbf{b}^{(\ell)}\right) \tag{1}$$

where AGGREGATE_Φ is the aggregation function induced by operator Φ (e.g., normalized neighborhood aggregation for GCN, attention-weighted aggregation for GAT), $\mathbf{W}^{(\ell)}$ are learnable weights, $\mathbf{b}^{(\ell)}$ is a bias term, and σ is a non-linear activation function (e.g., ReLU). The proposed framework extends this standard formulation through residual reuse mechanisms that either preserve intermediate node states within or across branches, or propagate graph-level summaries across successive computation blocks, as detailed in Section 4.

3.4 Graph-Level Representation Learning

After L layers of message passing, a readout function R aggregates node-level representations into a graph-level embedding $\mathbf{h}_\mathcal{G} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{h}_\mathcal{G} = R\left(\left\{\mathbf{h}_i^{(L)} : i = 1, \ldots, n\right\}\right) \tag{2}$$

Common readout functions include mean, max, and sum pooling. In our framework, graph-level representations may also be reused across successive computation blocks, enabling later stages to incorporate coarse whole-graph context in addition to local message-passing features.

3.5 Learning Objective

The graph-level representation is passed through a classifier (typically a linear layer followed by softmax) to produce the predicted label distribution:

$$\hat{\mathbf{y}} = \text{softmax}\left(\mathbf{W}_{\text{clf}}\mathbf{h}_{\mathcal{G}} + \mathbf{b}_{\text{clf}}\right) \tag{3}$$

where $\hat{\mathbf{y}} \in \mathbb{R}^C$ (for multi-class classification) is the predicted probability distribution over C classes, and $\mathbf{W}_{\text{clf}} \in \mathbb{R}^{C \times d_{\text{out}}}$, $\mathbf{b}_{\text{clf}} \in \mathbb{R}^C$ are classifier parameters.

The model is trained end-to-end by minimizing the cross-entropy loss function:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \tag{4}$$

where $y_{i,c} \in \{0, 1\}$ is the ground-truth label indicator (one-hot encoding) for graph $\mathcal{G}_i$ and class c, and $\hat{y}_{i,c}$ is the predicted probability for class c. For binary classification, this reduces to the binary cross-entropy loss. The parameters θ include all weights in the message passing layers, readout functions, cross-residual connections, and the classifier, which are optimized using gradient-based methods (e.g., Adam) with backpropagation.

4 Proposed Model

CR-GNN is formulated as a compact family of reusable graph classification architectures. All variants share the same input projection, stacked message-passing layers, global mean pooling, and linear graph classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches.

4.1 Operator Factory and Shared Backbone

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$. The operator factory instantiates one of three message-passing layers,

$$\Phi \in \{\text{GCNConv},\ \text{GATConv},\ \text{TransformerConv}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d, every branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A}))\,, \tag{5}$$

followed by L hidden layers and a readout

$$\mathbf{g} = \text{MeanPool}\Big(\mathbf{H}^{(L)}\Big)\,. \tag{6}$$

The final graph prediction is produced by

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{clf}}\,\text{Dropout}(\mathbf{g}) + \mathbf{b}_{\text{clf}}. \tag{7}$$

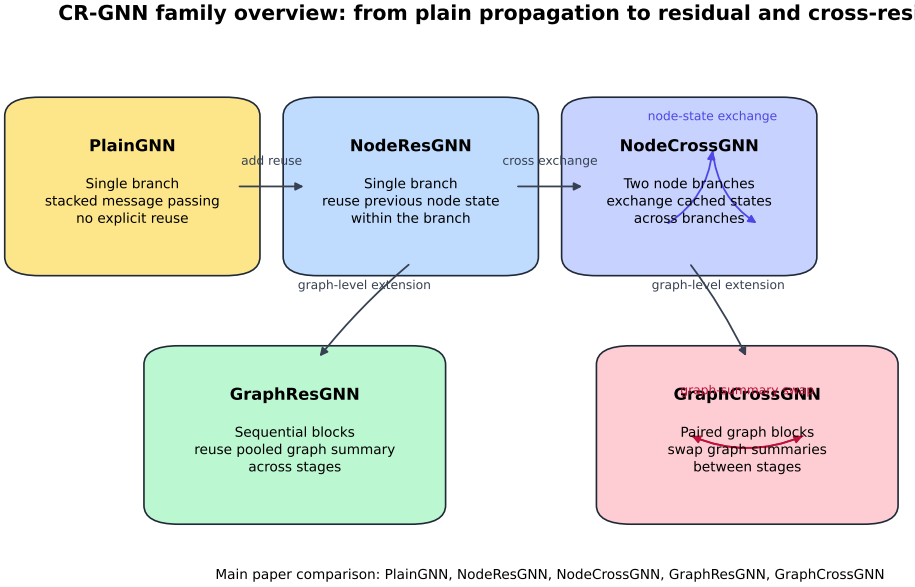

Figure 1: Overview of the CR-GNN family used in this paper. The main comparison moves from plain stacking to node-level residual reuse, node-level cross exchange, graph-level residual refinement, and graph-level cross-summary exchange.

The full implementation family contains six architectures, including the auxiliary GraphCondGNN variant. The main paper experiments focus on five core models, but all family members differ only in how they update $\mathbf{H}^{(\ell)}$ and how they reuse graph-level embeddings across blocks.

4.2 Single-Branch and Node-Level Residual Blocks

PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)}, \mathbf{A}\Big)\Big)\Big). \tag{8}$$

This model serves as the reference point for judging whether residual information flow is beneficial.

NodeResGNN. The first residual variant stores the previous hidden state within the same branch:

$$\mathbf{H}^{(\ell+1)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}\Big)\Big)\Big). \tag{9}$$

This mechanism is lightweight and preserves historical features without creating extra branches.

NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\Phi_\ell^{(1)}\Big(\mathbf{H}_1^{(\ell)} + \widetilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\Big)\Big)\Big), \tag{10}$$

$$\mathbf{H}_2^{(\ell+1)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\Phi_\ell^{(2)}\Big(\mathbf{H}_2^{(\ell)} + \widetilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\Big)\Big)\Big), \tag{11}$$

where $\widetilde{\mathbf{H}}_k^{(\ell)}$ is the cached hidden state from the opposite branch before its current update. The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \tag{12}$$

Compared with the single-branch residual block, NodeCrossGNN explicitly exchanges information between parallel pathways at every depth.

4.3 Graph-Level Residual Propagation

We further consider three higher-level architectures that compose several block models sequentially and pass graph embeddings across them.

GraphCondGNN. Two plain message-passing blocks are executed in sequence. The first block produces a graph embedding $\mathbf{g}^{(1)}$, which is added to node features and to the pooled graph representation inside the second block:

$$\mathbf{g}^{(2)} = B_2\Big(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(1)}\Big). \tag{13}$$

This introduces graph-level conditioning without intra-layer residual connections.

GraphResGNN. This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}\Big(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}\Big), \quad t \in \{1, 2\}. \tag{14}$$

Inside each block, the graph embedding is broadcast back to node states before the next graph convolution. The design can be viewed as repeated graph-level residual refinement.

GraphCrossGNN. The strongest graph-level interaction appears in the cross-graph variant. Four blocks are organized as two sequential pairs. Within each stage, the two block outputs are swapped and reused as the next stage's graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \Big(B_{2t-1}\Big(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}\Big),\ B_{2t}\Big(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}\Big)\Big), \tag{15}$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = \Big(\mathbf{g}_2^{(t)},\ \mathbf{g}_1^{(t)}\Big). \tag{16}$$

The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \tag{17}$$

In this variant, interaction occurs through exchanged pooled graph summaries rather than through direct node-tensor exchange.

4.4 Unified Training Objective

All model variants use the same graph classification objective:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c} \log \hat{y}_{i,c}, \tag{18}$$

Input: graph batch $(\mathbf{X}, \mathbf{A}, \mathbf{b})$, operator Φ, depth L, architecture type M
Instantiate message-passing blocks with operator Φ
if $M = \textsc{PlainGNN}$ **then**
    Apply one input projection and stack L message-passing layers
else if $M = \textsc{NodeResGNN}$ **then**
    Cache the previous hidden state and add it before each layer update
else if $M = \textsc{NodeCrossGNN}$ **then**
    Build two branches and inject the opposite branch's cached state at every depth
else if $M \in \{\textsc{GraphCondGNN}, \textsc{GraphResGNN}\}$ **then**
    Execute sequential block models and reuse the previous graph embedding as graph-level context
else if $M = \textsc{GraphCrossGNN}$ **then**
    Process block pairs, compute two graph embeddings, and swap them before the next pair
end if
Pool the final node representations with global mean pooling to obtain $\mathbf{g}$
Apply dropout and a linear classifier to produce logits $\hat{\mathbf{y}}$
Return: $(\hat{\mathbf{y}}, \mathbf{g})$

Figure 2: Pseudocode of the shared CR-GNN forward procedure

optimized with Adam. Dropout is applied after every message-passing update and once more before the classifier. The experiments in Section 6 therefore isolate architectural differences rather than changes in loss, optimizer, or data protocol.

4.5 Algorithmic View

Algorithm 2 summarizes the shared forward logic in pseudocode form rather than as a descriptive table.

4.6 Design Implications

This architecture family suggests three practical hypotheses that are later examined experimentally. First, residual reuse should matter more consistently than simply adding more branches, because every successful variant still relies on repeated feature reuse. Second, node-level cross residual and graph-level residual passing are not interchangeable: the former exchanges intermediate node representations, whereas the latter only re-injects pooled graph summaries. Third, the usefulness of cross-branch fusion depends on dataset scale and operator bias; it is therefore necessary to analyze accuracy, stability, and efficiency jointly rather than reporting only the single best score. Appendix A further gives a simplified linearized derivation showing why residual reuse can slow the collapse associated with over-smoothing.

5 Datasets

The original project evaluated a broad mixture of TU benchmarks, including molecular, vision, and generic graph-classification datasets. For the present submission target, that breadth alone is not an advantage. We therefore organize the evidence in two layers: a protein-oriented

main benchmark used for the biological narrative, and a broader full-suite benchmark used to characterize the architectural behavior of plain, residual, and cross-residual variants. This section describes both layers and clarifies how they are used in the paper.

5.1 Protein-Oriented Main Benchmark

Our main experiments use three graph classification datasets from the TU Dortmund collection accessed through PyTorch Geometric [42]:

- **PROTEINS**: graph-level protein classification
- **DD**: protein structure graph classification on much larger and sparser graphs
- **ENZYMES**: enzyme-oriented multi-class graph classification

This selection is deliberate. PROTEINS and DD are the strongest biological anchors already present in our codebase, while ENZYMES improves topic fit with only minimal engineering change because it remains within the same TUDataset interface. These three datasets drive the protein-oriented interpretation in the main text.

5.2 Full-Suite Structural Validation

To avoid over-claiming from only three datasets, we also evaluate the same model family on the complete local benchmark suite:

- **Supplementary biomolecular or chemical datasets**: MUTAG, AIDS, Mutagenicity
- **Out-of-scope control dataset**: MSRC_9

This broader suite is not introduced to strengthen topical fit. Instead, it serves a different purpose: it tests whether the observed behavior of plain, residual, and cross-residual information flow is stable across datasets with different graph scales, label structures, and semantic domains. Accordingly, the paper uses PROTEINS, DD, and ENZYMES for the main biological narrative, while the full-suite results are used as structural evidence about the architecture family.

5.3 Dataset Overview

PROTEINS PROTEINS contains graph representations of proteins, where nodes denote secondary structure elements and edges encode neighborhood relations between them. The task is binary graph classification. In the current data export, the dataset contains 1,113 graphs, 2 classes, and 3 input features per node. The average graph has 39.06 nodes and 72.82 edges, although the size variation is substantial.

DD DD is a protein structure dataset in which each graph corresponds to a protein and nodes represent amino acids or structure-related units linked by proximity-derived edges. The task is binary classification. DD is the largest and sparsest benchmark in the main package, with 1,178 graphs, 2 classes, and 89 input features. On average, graphs contain 284.32 nodes and 715.66 edges, with individual graphs reaching 5,748 nodes.

Table 1: Summary of the protein-oriented main benchmark datasets.

Dataset	# Graphs	# Classes	Feature Dim.	Avg. Nodes	Avg. Edges
PROTEINS	1113	2	3	39.06	72.82
DD	1178	2	89	284.32	715.66
ENZYMES	600	6	3	32.63	62.14

ENZYMES ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3 input features. It is attractive for the present paper because it is protein- and enzyme-oriented while still being easy to integrate into the existing graph-classification pipeline. The average graph has 32.63 nodes and 62.14 edges, and the class distribution is perfectly balanced across the six categories.

5.4 Dataset Statistics

Table 1 summarizes the key statistics of the protein-oriented main benchmark.

5.5 Data Loading

All datasets are loaded through the PyTorch Geometric `TUDataset` interface. Each graph is represented as a `Data` object containing node features `x`, graph connectivity `edge_index`, graph label `y`, and batch indices generated during mini-batch loading. If a graph is missing node attributes, the loader replaces them with a constant all-ones feature, ensuring that every benchmark can be processed by the same training code.

No additional graph augmentation, feature normalization, or edge-attribute modeling is applied in the final protocol. This is a deliberate choice: the goal is to isolate the effect of residual and cross-residual information flow rather than to combine it with a large preprocessing stack.

5.6 Splitting Protocol

The present V3 code no longer uses the earlier contiguous fold slicing scheme. Instead, all reported results, both for the protein-oriented benchmark and for the full suite, follow a stratified two-stage split:

- **Outer evaluation**: stratified 5-fold cross-validation at the graph level
- **Inner validation**: a stratified random validation subset sampled from the training fold with validation ratio 0.1

This change is important. Under weaker settings, several models collapsed toward majority-class behavior, which blurred architectural differences. The stratified outer fold construction and stratified inner validation split now provide a more reliable basis for early stopping and model comparison.

All experiments use a fixed global seed for reproducibility, while the inner train/validation split uses a fold-dependent seed so that every outer fold keeps a deterministic but distinct validation partition.

5.7 Evaluation Metrics

The primary evaluation metric is graph classification accuracy on the held-out test fold. For each method and dataset, we report the mean and standard deviation of the best test accuracy over the 5 stratified folds. We also track test loss, best epoch, parameter count, learning-rate history, gradient norms, and embedding statistics, which are used in the analysis scripts and TensorBoard logs.

5.8 Scope Note

Although the main benchmark is more biologically aligned than the generic benchmark mixture used earlier in the project, it still does not constitute a full integrative omics suite. Accordingly, the claims of this paper are intentionally conservative. We position the method as a biomolecular graph representation framework evaluated through a protein-oriented main benchmark and a broader structural validation suite, not as a complete omics-integration system or a direct solution to novel gene, domain, or motif discovery.

6 Experiments

This section reports the final V3 evaluation after the training protocol, hyperparameter schedule, and TensorBoard-backed logging pipeline were stabilized. The goal is to answer two questions. First, does cross-residual information flow provide a consistent benefit over plain stacking? Second, when compared against simpler residual reuse, where does the cross design help and where does it fail?

6.1 Experimental Settings

6.1.1 Model Set

We compare five architectures under the same `GCNConv` message-passing backbone:

1. **PlainGNN**: plain stacked propagation without explicit hidden-state reuse
2. **NodeResGNN**: single-branch node-level residual reuse
3. **NodeCrossGNN**: two-branch node-level cross residual exchange
4. **GraphResGNN**: graph-level residual refinement driven by pooled graph summaries
5. **GraphCrossGNN**: graph-level cross residual exchange across consecutive block pairs

This comparison isolates the effect of information-flow design while holding the operator family fixed.

6.1.2 Dataset Scope

The main biological discussion still centers on the protein-oriented datasets introduced in Section 5, namely PROTEINS, DD, and ENZYMES. However, the final implementation study also evaluates the complete local benchmark suite:

- **Topic-facing datasets**: PROTEINS, DD, ENZYMES
- **Supplementary biological or chemical datasets**: MUTAG, AIDS, Mutagenicity
- **Out-of-scope control dataset**: MSRC_9

This broader suite is used to characterize when cross-residual routing behaves like a genuine architectural advantage and when residual reuse remains preferable.

6.1.3 Data Protocol

All reported results use stratified 5-fold graph-level evaluation. Inside each training fold, we construct a stratified validation subset with validation ratio 0.1. Model selection is based on validation loss, and the best checkpoint is evaluated on the held-out test fold.

This protocol replaces the earlier weaker contiguous fold slicing used during exploratory runs. The stratified design matters because it reduces majority-class collapse and exposes architectural differences more reliably.

6.1.4 Optimization

All runs use Adam, validation-based early stopping, `ReduceLROnPlateau`, and gradient clipping. The shared defaults are:

- batch size = 32
- gradient clipping norm = 2.0
- scheduler factor = 0.5
- scheduler patience = 15
- minimum learning rate = 10^{-5}

Plain and residual baselines use a common configuration:

- epochs = 200
- patience = 60
- learning rate = 0.005
- weight decay = 10^{-4}
- dropout = 0.5
- hidden dimension = 64

Table 2: Full 5-fold benchmark results on the complete dataset suite. Numbers are mean best test accuracy ± standard deviation.

Model	MUTAG	PROTEINS	DD	ENZYMES
PlainGNN	0.7236 ± 0.0486	0.7053 ± 0.0271	0.7165 ± 0.0179	0.2450 ± 0.0407
NodeResGNN	0.7343 ± 0.0734	0.6945 ± 0.0509	**0.7207 ± 0.0259**	0.2650 ± 0.0661
NodeCrossGNN	**0.7397 ± 0.0676**	0.6972 ± 0.0430	0.7198 ± 0.0178	0.3000 ± 0.0580
GraphResGNN	0.7290 ± 0.0585	**0.7223 ± 0.0367**	0.6934 ± 0.0458	**0.3117 ± 0.0746**
GraphCrossGNN	0.7129 ± 0.0539	0.6864 ± 0.0465	0.7097 ± 0.0309	0.3017 ± 0.0750

Table 3: Full-suite continuation table for the remaining datasets.

Model	MSRC_9	AIDS	Mutagenicity
PlainGNN	0.8868 ± 0.0144	0.8455 ± 0.0257	0.7814 ± 0.0235
NodeResGNN	0.9005 ± 0.0097	0.8690 ± 0.0354	0.7897 ± 0.0139
NodeCrossGNN	0.8913 ± 0.0169	0.8735 ± 0.0412	**0.8033 ± 0.0159**
GraphResGNN	0.9003 ± 0.0234	**0.9160 ± 0.0121**	0.8022 ± 0.0220
GraphCrossGNN	**0.9049 ± 0.0334**	0.8710 ± 0.0412	0.8029 ± 0.0182

- hidden layers = 4

Cross-residual models use the stronger cross-specific V3 protocol:

- default cross setting: epochs = 240, patience = 80, learning rate = 0.003, weight decay = 5×10^{-5}, dropout = 0.3, hidden dimension = 64, hidden layers = 4
- PROTEINS / NodeCrossGNN: dropout = 0.2
- DD / NodeCrossGNN: hidden layers = 3
- DD / GraphCrossGNN: learning rate = 0.002, dropout = 0.2

6.1.5 Reproducibility

All final runs were executed with the same V3 code path, stratified 5-fold evaluation, and deterministic seed control. The experiments were run on a single NVIDIA RTX 3090 GPU with 24GB memory. Each completed run writes a structured JSON summary to the project `logs/` directory, and optional TensorBoard traces are written to `runs/`. The repository also includes batch execution, summarization, and report-generation scripts so that the final tables in this paper can be regenerated directly from the stored experiment outputs.

6.2 Full-Suite Results

Tables 2–4 summarize the final full-suite benchmark over all seven available datasets.

The full-suite winner pattern is immediately informative. NodeCrossGNN wins on MUTAG and Mutagenicity, GraphCrossGNN wins on MSRC_9, GraphResGNN wins on PROTEINS, ENZYMES, and AIDS, and NodeResGNN wins on DD. PlainGNN does not win any dataset in the final suite.

Table 4: Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
MUTAG	NodeCrossGNN	0.7397	88.2	34434
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MSRC_9	GraphCrossGNN	0.9049	38.8	71976
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

6.3 Main Findings

Residual reuse remains the strongest default family. Residual variants win four of the seven datasets in the full suite: PROTEINS, DD, ENZYMES, and AIDS. This makes residual reuse the safest default baseline under the stabilized protocol. In particular, GraphResGNN is best on PROTEINS (0.7223±0.0367), ENZYMES (0.3117±0.0746), and AIDS (0.9160±0.0121), while NodeResGNN is best on DD (0.7207 ± 0.0259).

Cross-residual design is selectively strong, not universally dominant. Cross variants still win three datasets in the full suite. NodeCrossGNN is best on MUTAG (0.7397 ± 0.0676) and Mutagenicity (0.8033±0.0159), while GraphCrossGNN is best on MSRC_9 (0.9049± 0.0334). More importantly, the best cross variant beats PlainGNN on six of the seven datasets. Therefore, cross routing is clearly more than an implementation curiosity: it is a useful alternative information-flow mechanism. However, it beats the best residual baseline on only three datasets, so the final evidence does not support the stronger claim that cross universally outperforms residual reuse.

Node-level cross is more reliable than graph-level cross. Across the full suite, NodeCrossGNN outperforms NodeResGNN on five of the seven datasets, whereas GraphCrossGNN outperforms GraphResGNN on only three. This suggests that exchanging intermediate node-level states is the more robust cross-residual design in the present implementation, while graph-level cross routing is more dataset-sensitive.

Plain stacking is no longer a credible final baseline. PlainGNN remains competitive on PROTEINS and DD, but it never wins a dataset in the final suite and is usually surpassed by either a residual or a cross-residual variant. This is important for the paper narrative: the meaningful comparison is no longer cross versus plain, but cross versus the stronger residual family.

6.4 Topic-Facing Biological Interpretation

For the target venue, the most relevant evidence still comes from PROTEINS, DD, and ENZYMES.

On **PROTEINS**, GraphResGNN is clearly strongest, indicating that repeated graph-level residual refinement is the most effective way to preserve useful global structure. The best cross model here is NodeCrossGNN at 0.6972 ± 0.0430, which improves slightly over NodeResGNN but remains below both PlainGNN and GraphResGNN.

On **DD**, NodeResGNN and NodeCrossGNN are nearly tied, with NodeResGNN retaining a narrow lead. This indicates that node-level hidden-state reuse is important on large sparse protein graphs, but cross exchange does not produce a decisive improvement over simpler residual reuse.

On **ENZYMES**, cross variants become more attractive. GraphCrossGNN reaches 0.3017 ± 0.0750 and NodeCrossGNN reaches 0.3000 ± 0.0580, both clearly above PlainGNN at 0.2450 ± 0.0407. Even so, GraphResGNN still remains the best model. Therefore, ENZYMES is the cleanest topic-facing dataset for arguing that cross-residual pathways can recover useful information that plain stacking loses, but not for arguing that cross should replace residual baselines outright.

6.5 Ablation Conclusions

The final ablation story is therefore precise:

- **Plain → Cross**: cross-residual routing is usually beneficial relative to plain stacking
- **Residual → Cross**: the gain is conditional and dataset-dependent
- **Node-level versus graph-level**: node-level cross is more stable, while graph-level cross achieves stronger wins only on selected datasets

This is the strongest claim that the current evidence supports. Cross-residual design should be presented as a meaningful and reusable architectural design space, not as a universal winner over residual reuse.

6.6 Analysis Assets

To support interpretation beyond scalar accuracy, the current V3 pipeline records:

- TensorBoard traces for loss, accuracy, learning rate, gradient norm, embedding norms, and logit statistics
- exported artifacts for graph embeddings, logits, predictions, and confusion-matrix reconstruction
- per-fold JSON logs containing parameter counts, optimization trajectories, and best-epoch summaries

These materials make it possible to distinguish genuine structural gains from accidental improvements caused by unstable training dynamics.

6.7 Current Limitations

The experiments remain structure-centric rather than fully omics-integrative. Accordingly, the strongest defensible claim of the paper is that CR-GNN is an effective biomolecular graph representation architecture with dataset-dependent cross-residual gains. The current evidence does not yet justify a broader claim about integrative omics prediction or direct discovery of novel genes, domains, or motifs.

7 Conclusion

This paper reframed the cross-residual GNN study around biomolecular graph representation learning rather than generic benchmark breadth. The resulting picture is more precise than the earlier exploratory narrative. Cross-residual interaction is a meaningful architectural tool, but its benefit is conditional: in the final seven-dataset suite, residual variants are the strongest default family, while cross-residual variants remain selectively competitive rather than universally dominant.

The main practical outcome is therefore not that one cross-residual model dominates every benchmark. Instead, the contribution is a reusable design principle and a controlled architecture family that makes these trade-offs visible. In the final results, residual models win on PROTEINS, DD, ENZYMES, and AIDS, while cross-residual models win on MUTAG, MSRC_9, and Mutagenicity. Just as importantly, the best cross variant improves over the plain baseline on six of the seven datasets. This means cross-residual routing is useful and nontrivial, but the strongest defensible claim is a dataset-dependent advantage, not a universal superiority claim over residual reuse.

For the topic-facing biological interpretation, the most relevant datasets remain PROTEINS, DD, and ENZYMES. On these benchmarks, GraphResGNN is strongest on PROTEINS and ENZYMES, whereas NodeResGNN is strongest on DD. Cross variants still provide useful gains over plain stacking, especially on ENZYMES, but they do not replace residual baselines as the default point of reference. This is the right level of claim for the current evidence.

There are three immediate directions for future work. First, the topic fit can be strengthened by adding datasets that are closer to protein function prediction or to genuine multi-signal biological inference, such as protein-protein interaction datasets with ontology supervision. Second, the current graph-classification backbone can be extended toward node-level biological prediction tasks so that cross-residual mechanisms can be tested under richer supervision. Third, the architecture-level findings reported here can be paired with stronger biological features, including sequence-derived embeddings or other omics-derived signals, to move beyond structure-only benchmarks.

A Why Residual Reuse Can Slow Over-smoothing

This appendix provides a simplified derivation that explains why residual reuse can mitigate over-smoothing. The goal is not to claim a full theorem for the nonlinear models in Section 4, but to show that the proposed residual pathways change the effective propagation filter from a single high-order power to a lower-order polynomial mixture.

A.1 Linearized Plain Propagation

Consider the standard linearized message-passing update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}\mathbf{H}^{(\ell)}, \tag{19}$$

where $\mathbf{S}$ is a normalized graph propagation operator and $\mathbf{H}^{(0)} = \mathbf{X}$. Ignoring nonlinearities and trainable weights yields

$$\mathbf{H}^{(\ell)} = \mathbf{S}^{\ell}\mathbf{X}. \tag{20}$$

If $\mathbf{S}$ is diagonalizable as $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}$, then

$$\mathbf{H}^{(\ell)} = \mathbf{U}\mathbf{\Lambda}^{\ell}\mathbf{U}^{-1}\mathbf{X}. \tag{21}$$

For normalized propagation, the dominant eigenvalue is typically close to 1, while the remaining eigenvalues satisfy $|\lambda_i| < 1$. As ℓ grows, the factors λ_i^{ℓ} suppress non-dominant components, so node embeddings gradually collapse toward a low-dimensional smooth subspace. This is the standard over-smoothing effect: nodes become increasingly difficult to distinguish because repeated propagation attenuates the higher-frequency components that encode local differences.

A.2 Node-Level Residual Reuse

Now consider a residual reuse update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}\big(\mathbf{H}^{(\ell)} + \beta\mathbf{H}^{(\ell-1)}\big), \tag{22}$$

where $\beta \geq 0$ controls the strength of the reused hidden state. The implementation studied in this paper corresponds to $\beta = 1$.

Define $\mathbf{H}^{(\ell)} = P_{\ell}(\mathbf{S})\mathbf{X}$ with $P_0(\mathbf{S}) = \mathbf{I}$ and $P_1(\mathbf{S}) = \mathbf{S}$. Then Eq. (22) implies the polynomial recurrence

$$P_{\ell+1}(\mathbf{S}) = \mathbf{S}\big(P_{\ell}(\mathbf{S}) + \beta P_{\ell-1}(\mathbf{S})\big). \tag{23}$$

The first few terms are

$$P_1(\mathbf{S}) = \mathbf{S}, \tag{24}$$
$$P_2(\mathbf{S}) = \mathbf{S}^2 + \beta\mathbf{S}, \tag{25}$$
$$P_3(\mathbf{S}) = \mathbf{S}^3 + 2\beta\mathbf{S}^2, \tag{26}$$
$$P_4(\mathbf{S}) = \mathbf{S}^4 + 3\beta\mathbf{S}^3 + \beta^2\mathbf{S}^2. \tag{27}$$

Therefore, residual reuse no longer applies only the single power $\mathbf{S}^{\ell}$; instead, it produces a mixture of lower-order and higher-order propagation terms. In spectral coordinates, each eigencomponent evolves as

$$z_i^{(\ell+1)} = \lambda_i\big(z_i^{(\ell)} + \beta z_i^{(\ell-1)}\big), \tag{28}$$

which is fundamentally different from the plain recurrence $z_i^{(\ell)} = \lambda_i^{\ell} z_i^{(0)}$. The plain model attenuates every non-dominant mode with a pure monomial factor, whereas the residual model preserves a polynomial combination of powers of λ_i. As a result, informative mid-frequency components are not suppressed as aggressively at the same depth.

A.3 Implication for Over-smoothing

The derivation above suggests the following interpretation. Plain stacking behaves like a repeated low-pass filter whose order increases monotonically with depth. Residual reuse changes that filter into a polynomial response that still smooths the graph, but preserves more low-order propagation information from earlier layers. This does not eliminate smoothing, yet it slows the rate at which node representations collapse to an almost indistinguishable subspace. In other words, residual reuse can improve depth tolerance because it reduces the dominance of the highest propagation order.

A.4 Graph-Level Residual Passing

The same intuition extends to graph-level residual propagation. In the graph-conditioned variants, a pooled summary from the previous block is broadcast back to the current block, so the hidden state is no longer determined only by repeated applications of $\mathbf{S}$. A simplified affine form is

$$\mathbf{H}_t^{(\ell+1)} = \mathbf{S}\mathbf{H}_t^{(\ell)} + \mathbf{1}\big(\mathbf{g}_{t-1}\big)^\top, \tag{29}$$

where $\mathbf{g}_{t-1}$ is the preceding graph summary. The additive term injects a lower-order global context at every block and prevents the representation from being governed solely by deeper powers of the propagation operator. This offers a second pathway for delaying over-smoothing, although the empirical sections show that graph-level reuse is more sensitive to dataset structure and computational cost than the lighter node-level residual design.

A.5 Scope of the Derivation

This appendix uses a linearized analysis and therefore abstracts away nonlinear activations, dropout, optimizer dynamics, and operator-specific details. It should be read as an explanatory mechanism rather than as a complete proof. Still, it aligns with the empirical observation that node-level residual reuse remains more stable than plain stacking as depth increases, and it provides a compact theoretical rationale for why reusing earlier states can alleviate over-smoothing.

Table 5: Full 5-fold benchmark results on the complete dataset suite. Numbers are mean best test accuracy ± standard deviation.

Model	MUTAG	PROTEINS	DD	ENZYMES
PlainGNN	0.7236 ± 0.0486	0.7053 ± 0.0271	0.7165 ± 0.0179	0.2450 ± 0.0407
NodeResGNN	0.7343 ± 0.0734	0.6945 ± 0.0509	**0.7207 ± 0.0259**	0.2650 ± 0.0661
NodeCrossGNN	**0.7397 ± 0.0676**	0.6972 ± 0.0430	0.7198 ± 0.0178	0.3000 ± 0.0580
GraphResGNN	0.7290 ± 0.0585	**0.7223 ± 0.0367**	0.6934 ± 0.0458	**0.3117 ± 0.0746**
GraphCrossGNN	0.7129 ± 0.0539	0.6864 ± 0.0465	0.7097 ± 0.0309	0.3017 ± 0.0750

Table 6: Full-suite continuation table for the remaining datasets.

Model	MSRC_9	AIDS	Mutagenicity
PlainGNN	0.8868 ± 0.0144	0.8455 ± 0.0257	0.7814 ± 0.0235
NodeResGNN	0.9005 ± 0.0097	0.8690 ± 0.0354	0.7897 ± 0.0139
NodeCrossGNN	0.8913 ± 0.0169	0.8735 ± 0.0412	**0.8033 ± 0.0159**
GraphResGNN	0.9003 ± 0.0234	**0.9160 ± 0.0121**	0.8022 ± 0.0220
GraphCrossGNN	**0.9049 ± 0.0334**	0.8710 ± 0.0412	0.8029 ± 0.0182

B Complete Benchmark Tables

For completeness, we reproduce the final full-suite benchmark tables in the appendix. These are the same results discussed in Section 6, but collecting them here makes the full comparison easier to inspect during review.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[2] X. Bresson and T. Laurent, "Residual gated graph convnets," *arXiv preprint arXiv:1711.07553*, 2017.

[3] L. Zhao and L. Akoglu, "Pairnorm: Tackling over-smoothing in gns," *arXiv preprint arXiv:2009.10698*, 2020.

[4] T. Cai, J. Luo, S. Wang, K. Zhang, Y. Tian, L. Yang, Z. Li, and K. Weinberger, "Graphnorm: A principled approach to accelerating graph neural network training," in *International Conference on Machine Learning*, 2021, pp. 1277–1287.

[5] Y. Rong, W. Huang, T. Xu, and J. Huang, "Dropedge: Towards deep graph convolutional networks on node classification," *arXiv preprint arXiv:1907.10903*, 2019.

[6] K. Xu, C. Li, Y. Tian, X. Du, J. Leskovec, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *International Conference on Machine Learning*, 2018, pp. 5449–5458.

Table 7: Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
MUTAG	NodeCrossGNN	0.7397	88.2	34434
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MSRC_9	GraphCrossGNN	0.9049	38.8	71976
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

[7] M. Gori, G. Monfardini, and F. Scarselli, "A new model for learning in graph domains," *Proceedings of IJCNN*, 2005.

[8] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *International Conference on Machine Learning*, 2017, pp. 1263–1272.

[9] D. K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams, "Convolutional networks on graphs for learning molecular fingerprints," in *Advances in neural information processing systems*, 2015, pp. 2224–2232.

[10] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Advances in neural information processing systems*, 2018, pp. 4800–4810.

[11] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," *arXiv preprint arXiv:1904.08082*, 2019.

[12] C. Cangea, P. Veličković, N. Jovanović, T. Kipf, and P. Liò, "Towards sparse hierarchical graph classifiers," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2018, pp. 165–176.

[13] C. Liu, X. Wang, Y. Li, Y. Wang, and J. Gao, "Graph pooling for graph neural networks," in *International Joint Conference on Artificial Intelligence*, 2023, pp. 4254–4261.

[14] Z. Li, H. Chen, Z. Zeng, J. He, X. Mao, N. Lv, and L. Chen, "Graph pooling for graph-level representation learning: a survey," *Artificial Intelligence Review*, vol. 57, no. 8, pp. 1–51, 2024.

[15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.

[16] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, 2017, pp. 1024–1034.

[17] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations (ICLR)*, 2019.

[18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[19] M. Zhang, Y. Liu, Q. Chen, and S. Pan, "Personalized graph neural networks with attention mechanism," *arXiv preprint arXiv:1910.08887*, 2019.

[20] V. P. Dwivedi and X. Bresson, "A generalization of transformer networks to graphs," *arXiv preprint arXiv:2012.09699*, 2020.

[21] D. Beaini, S. Passaro, E. Létouzey, J. Domke, S. Erdos, P. Lió, P. Veličković, and M. M. Bronstein, "Directional graph networks," *arXiv preprint arXiv:2010.02863*, 2020.

[22] E. Rossi, F. Frasca, F. M. Bianchi, N. Alletto, and P. Lio, "Edge directionality improves learning on heterophilic graphs," in *International Conference on Learning Representations (ICLR)*, 2023.

[23] F. M. Bianchi, F. Grattarola, and C. Alippi, "Graph neural networks with convolutional arma filters," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[24] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, G. Steeg, and A. Galstyan, "Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing," in *International Conference on Machine Learning*, 2019, pp. 33–42.

[25] A. Gravina, N. Alletto, A. Rossi, F. M. Bianchi, and P. Lio, "Anti-symmetric dgn: A stable architecture for deep graph networks," *arXiv preprint arXiv:2210.09789*, 2022.

[26] G. Li, E. Muller, A. Thabet, and B. Ghanem, "Deeper insights into graph convolutional networks for semi-supervised learning," *arXiv preprint arXiv:1809.02584*, 2018.

[27] Y. Chen, X. Wei, P. Xu, X. Wang, P. Luo, Z. Li, and J. Tang, "Revisiting over-smoothing in deep gcns," *arXiv preprint arXiv:2003.13663*, 2020.

[28] S. Li, M. Zhang, M. Jin, H. Zhang, R. Ying, Z. Li, and J. Yan, "A non-asymptotic analysis of oversmoothing in graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2023.

[29] J. Topping, F. Di Giovanni, B. Chamberlain, M. Bronstein, D. Vendrig, and P. Lio, "Understanding over-squashing and bottlenecks on graphs via curvature," *arXiv preprint arXiv:2111.14522*, 2021.

[30] U. Alon and E. Yahav, "On the bottleneck of graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2021.

[31] Y. Li, X. Wang, Z. Wang, and J. Li, "Learning graph normalization for graph neural networks," *Neurocomputing*, vol. 486, pp. 140–151, 2022.

[32] G. Li, E. Müller, A. Thabet, and B. Ghanem, "Training graph neural networks with 1000 layers," in *International Conference on Machine Learning*, 2021, pp. 6403–6413.

[33] K. Oono and T. Suzuki, "Simple and deep graph convolutional networks," *arXiv preprint arXiv:2006.13604*, 2020.

[34] G. Wang, I. Balazevic, M. Znis, M. Gitta, Y. Kwon, M. Seeger, X. Li, L. De Raedt, and M. Welling, "Multi-hop attention graph neural networks," in *International Joint Conference on Artificial Intelligence*, 2021, pp. 2112–2120.

[35] X. Sun, R. Wang, L. Cui, X. Kong, and S. Li, "Attention-based graph neural networks: a survey," *Artificial Intelligence Review*, vol. 56, no. 11, pp. 13 493–13 539, 2023.

[36] J. Cai, X. Zheng, C. Luo, C. Yang, and E. Santus, "Understanding graph embedding methods and their applications," *arXiv preprint arXiv:2012.08019*, 2020.

[37] H. Du, Y. Zhang, P. Zhang, B. Gu, X. Wei, J. Guo, A. Li, Z. Wang, T. Ma *et al.*, "Densegnn: universal and scalable deeper graph neural networks for high-performance property prediction in crystals and molecules," *npj Computational Materials*, vol. 10, no. 1, p. 304, 2024.

[38] H. Van Pham, U. The, T. Do, and A. Nguyen, "Hierarchical pooling in graph neural networks to enhance graph classification," *Applied Sciences*, vol. 11, no. 17, p. 8164, 2021.

[39] H. Gao and S. Ji, "Graph u-net," *arXiv preprint arXiv:1905.05178*, 2019.

[40] J. Klicpera, A. Bojchevski, and S. Günnemann, "Combining label propagation and simple models out-performs graph neural networks," *arXiv preprint arXiv:1810.05997*, 2018.

[41] L. Zhao, Z. Leng, Y. Chen, and L. Akoglu, "Tackling over-smoothing for general graph learning," *arXiv preprint arXiv:2008.09864*, 2020.

[42] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," *arXiv preprint arXiv:1903.02428*, 2019.